paper

April 23, 2026

1 LendingClub PD notebook — paper-style logit + Cox

This notebook is intentionally **simple** and **paper-mimicking**.

It follows the spirit of the LendingClub study by **Serrano-Cinca, Gutiérrez-Nieto, and López-Palacios (2015)**: 1. use borrower and loan characteristics available at origination, 2. fit a **standard frequency-based model** for default prediction, 3. fit a **Cox proportional hazards model** as the time-to-default extension, 4. compare them on a **supported fixed long-term horizon**, 5. hand teammates one clean PD output table.

1.1 What this notebook is for

- **Main handoff PD**: 12-month **logit PD**
- **Simple long-term PD**: compare **24-month logit** vs **24-month Cox-implied PD**
- **Chosen long-term PD**: automatically pick the better of the two on validation Brier score

1.2 What this notebook is not trying to do

- no 36/60-only segmentation
- no full CECL engine
- no giant model zoo
- no unsupported 36/60 month backtests on recent vintages

[1]:
```
import warnings
warnings.filterwarnings("ignore")

from pathlib import Path
import subprocess
import sys
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import (
```

```
    roc_auc_score,
    average_precision_score,
    brier_score_loss,
    roc_curve,
)
from sklearn.calibration import calibration_curve

try:
    from lifelines import KaplanMeierFitter, CoxPHFitter
    from lifelines.utils import concordance_index
except ModuleNotFoundError:
    print("Installing lifelines...")
    subprocess.check_call([sys.executable, "-m", "pip", "install", "lifelines",␣
 ↪"-q"])
    from lifelines import KaplanMeierFitter, CoxPHFitter
    from lifelines.utils import concordance_index
```

1.3 1. Runtime controls

This is the only place you should need to change first.

[2]:
```
RANDOM_STATE = 42

# Turn sampling on only if runtime becomes annoying on your machine.
USE_SAMPLE = False
SAMPLE_SIZE = 250_000

# We keep a clean, supported setup:
# - 12m frequency PD
# - 24m long-term comparison between standard logit and Cox
SHORT_HORIZON = 12
LONG_HORIZON = 24

# Intertemporal validation in the style of the paper:
# train on older vintages, validate on 2015, test on 2016.
TRAIN_END = "2014-12-31"
VALID_START, VALID_END = "2015-01-01", "2015-12-31"
TEST_START, TEST_END = "2016-01-01", "2016-12-31"

DATA_DIR = Path("data")
RAW_DIR = DATA_DIR / "raw"
OUTPUT_DIR = DATA_DIR / "processed_paper_style_pd"
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

print("Output directory:", OUTPUT_DIR.resolve())
```

```
Output directory:
/Users/pattonpatton/Desktop/RiskProject/Risk_Management_Project/Time
```

```
Based/data/processed_paper_style_pd
```

1.4 2. Find and load the LendingClub file

[3]:
```
def find_accepted_file():
    candidates = [
        Path("data/raw/lendingclub/accepted_2007_to_2018Q4.csv.gz"),
        Path("data/raw/accepted_2007_to_2018Q4.csv.gz"),
        Path("accepted_2007_to_2018Q4.csv.gz"),
        Path("/mnt/data/accepted_2007_to_2018Q4.csv.gz"),
        Path("/mnt/data/data/raw/lendingclub/accepted_2007_to_2018Q4.csv.gz"),
    ]
    for path in candidates:
        if path.exists():
            return path
    raise FileNotFoundError(
        "Could not find accepted_2007_to_2018Q4.csv.gz. Put it in data/raw/
 ↪lendingclub/ "
        "or edit find_accepted_file()."
    )

accepted_path = find_accepted_file()
print("Using raw file:", accepted_path)
```

```
Using raw file: data/raw/lendingclub/accepted_2007_to_2018Q4.csv.gz
```

[4]:
```
key_columns = [
    "id",
    "loan_status",
    "issue_d",
    "term",
    "grade",
    "int_rate",
    "loan_amnt",
    "installment",
    "emp_length",
    "home_ownership",
    "annual_inc",
    "verification_status",
    "purpose",
    "dti",
    "delinq_2yrs",
    "inq_last_6mths",
    "open_acc",
    "pub_rec",
    "revol_bal",
    "revol_util",
    "total_acc",
```

```python
    "mort_acc",
    "pub_rec_bankruptcies",
    "fico_range_low",
    "fico_range_high",
    "earliest_cr_line",
    "last_pymnt_d",
    "last_credit_pull_d",
]

df_raw = pd.read_csv(
    accepted_path,
    usecols=lambda c: c in key_columns,
    low_memory=False,
    compression="gzip",
)

if USE_SAMPLE:
    df_raw = df_raw.sample(min(SAMPLE_SIZE, len(df_raw)),␣
 ↪random_state=RANDOM_STATE).copy()

print("Raw shape:", df_raw.shape)
display(df_raw.head(3))
```

```
Raw shape: (2260701, 28)

         id  loan_amnt        term  int_rate  installment grade emp_length  \
0  68407277     3600.0   36 months     13.99       123.03     C  10+ years
1  68355089    24700.0   36 months     11.99       820.28     C  10+ years
2  68341763    20000.0   60 months     10.78       432.66     B  10+ years

  home_ownership  annual_inc verification_status  … inq_last_6mths open_acc  \
0       MORTGAGE     55000.0        Not Verified  …            1.0      7.0
1       MORTGAGE     65000.0        Not Verified  …            4.0     22.0
2       MORTGAGE     63000.0        Not Verified  …            0.0      6.0

  pub_rec  revol_bal  revol_util total_acc  last_pymnt_d  last_credit_pull_d  \
0     0.0     2765.0        29.7      13.0      Jan-2019            Mar-2019
1     0.0    21470.0        19.2      38.0      Jun-2016            Mar-2019
2     0.0     7869.0        56.2      18.0      Jun-2017            Mar-2019

   mort_acc  pub_rec_bankruptcies
0       1.0                   0.0
1       4.0                   0.0
2       5.0                   0.0

[3 rows x 28 columns]
```

1.5 3. Clean the loan-level data

This keeps the portfolio broad.
No 36/60 restriction is imposed.

[5]:
```
df = df_raw.copy()

for c in ["issue_d", "earliest_cr_line", "last_pymnt_d", "last_credit_pull_d"]:
    df[c] = pd.to_datetime(df[c], format="%b-%Y", errors="coerce")

status = df["loan_status"].fillna("").astype(str)

# Keep the default definition simple and consistent.
default_mask = status.str.contains("Charged Off|Default", case=False,␣
 ↪regex=True)
payoff_mask = status.str.contains("Fully Paid", case=False, regex=False) &␣
 ↪(~default_mask)

df["default_event"] = default_mask.astype(int)
df["payoff_event"] = payoff_mask.astype(int)
df["event_type"] = np.select(
    [df["default_event"] == 1, df["payoff_event"] == 1],
    [1, 2],
    default=0,
)

df["term_num"] = (
    df["term"].astype(str).str.extract(r"(\d+)", expand=False).astype(float)
)

# Strip percentage signs where needed.
df["int_rate"] = (
    df["int_rate"].astype(str).str.replace("%", "", regex=False)
)
df["int_rate"] = pd.to_numeric(df["int_rate"], errors="coerce")

df["fico_avg"] = (df["fico_range_low"] + df["fico_range_high"]) / 2
df["credit_history_years"] = (
    (df["issue_d"] - df["earliest_cr_line"]).dt.days / 365.25
)

df["end_date"] = df["last_pymnt_d"]
missing_end = df["end_date"].isna()
df.loc[missing_end, "end_date"] = df.loc[missing_end, "last_credit_pull_d"]

def month_diff(start, end):
    return (end.dt.year - start.dt.year) * 12 + (end.dt.month - start.dt.month)
```

```
df = df[df["issue_d"].notna()].copy()
df = df[df["end_date"].notna()].copy()

df["duration_months"] = month_diff(df["issue_d"], df["end_date"]).clip(lower=1)

numeric_cols = [
    "int_rate",
    "loan_amnt",
    "installment",
    "annual_inc",
    "dti",
    "delinq_2yrs",
    "inq_last_6mths",
    "open_acc",
    "pub_rec",
    "revol_bal",
    "revol_util",
    "total_acc",
    "mort_acc",
    "pub_rec_bankruptcies",
    "fico_avg",
    "credit_history_years",
    "term_num",
]

for c in numeric_cols:
    df[c] = pd.to_numeric(df[c], errors="coerce")

# To keep the long-horizon comparison honest, use only vintages whose 24m␣
↪outcome can be known.
# Later vintages can still be scored later if needed, but they are not used in␣
↪this paper-style
# train/valid/test comparison.
df_model = df[df["issue_d"] <= pd.Timestamp(TEST_END)].copy()

print("Clean modeling shape:", df_model.shape)
print("Default share:", round(df_model["default_event"].mean(), 4))
print("Payoff share :", round(df_model["payoff_event"].mean(), 4))
display(df_model[["id", "issue_d", "loan_status", "default_event",␣
↪"payoff_event", "duration_months"]].head())
```

```
Clean modeling shape: (1321847, 36)
Default share: 0.1674
Payoff share : 0.6817
```

```
         id    issue_d loan_status  default_event  payoff_event  \
0  68407277 2015-12-01  Fully Paid              0             1
1  68355089 2015-12-01  Fully Paid              0             1
```

```
2  68341763 2015-12-01  Fully Paid              0             1
3  66310712 2015-12-01     Current              0             0
4  68476807 2015-12-01  Fully Paid              0             1

   duration_months
0               37
1                6
2               18
3               38
4                7
```

1.6 4. Paper-style intertemporal split

This mirrors the paper's idea of older training data and a later holdout period.

[6]:
```
train_loans = df_model[df_model["issue_d"] <= pd.Timestamp(TRAIN_END)].copy()
valid_loans = df_model[
    (df_model["issue_d"] >= pd.Timestamp(VALID_START)) &
    (df_model["issue_d"] <= pd.Timestamp(VALID_END))
].copy()
test_loans = df_model[
    (df_model["issue_d"] >= pd.Timestamp(TEST_START)) &
    (df_model["issue_d"] <= pd.Timestamp(TEST_END))
].copy()

split_table = pd.DataFrame([
    {"split": "train", "n_loans": len(train_loans), "issue_start":␣
 ↪train_loans["issue_d"].min(), "issue_end": train_loans["issue_d"].max()},
    {"split": "valid", "n_loans": len(valid_loans), "issue_start":␣
 ↪valid_loans["issue_d"].min(), "issue_end": valid_loans["issue_d"].max()},
    {"split": "test", "n_loans": len(test_loans), "issue_start":␣
 ↪test_loans["issue_d"].min(), "issue_end": test_loans["issue_d"].max()},
])
display(split_table)
```

```
   split  n_loans issue_start  issue_end
0  train   466345  2007-06-01 2014-12-01
1  valid   421095  2015-01-01 2015-12-01
2   test   434407  2016-01-01 2016-12-01
```

1.7 5. Build fixed-horizon labels

This is the **frequency-based PD** piece from the notes. - 12-month PD is the main handoff output - 24-month PD is the simple long-term frequency benchmark

[7]:
```
def make_horizon_label(loans, horizon):
    out = loans.copy()
    out["known_horizon_outcome"] = (
```

```
        (out["duration_months"] >= horizon) |
        ((out["event_type"].isin([1, 2])) & (out["duration_months"] <= horizon))
    )
    out = out[out["known_horizon_outcome"]].copy()
    out[f"default_within_{horizon}m"] = (
        (out["default_event"] == 1) & (out["duration_months"] <= horizon)
    ).astype(int)
    return out

train_12 = make_horizon_label(train_loans, SHORT_HORIZON)
valid_12 = make_horizon_label(valid_loans, SHORT_HORIZON)
test_12 = make_horizon_label(test_loans, SHORT_HORIZON)

train_24 = make_horizon_label(train_loans, LONG_HORIZON)
valid_24 = make_horizon_label(valid_loans, LONG_HORIZON)
test_24 = make_horizon_label(test_loans, LONG_HORIZON)

label_summary = pd.DataFrame([
    {"dataset": "train_12", "n": len(train_12), "default_rate":␣
 ↪train_12[f"default_within_{SHORT_HORIZON}m"].mean()},
    {"dataset": "valid_12", "n": len(valid_12), "default_rate":␣
 ↪valid_12[f"default_within_{SHORT_HORIZON}m"].mean()},
    {"dataset": "test_12",  "n": len(test_12),  "default_rate":␣
 ↪test_12[f"default_within_{SHORT_HORIZON}m"].mean()},
    {"dataset": "train_24", "n": len(train_24), "default_rate":␣
 ↪train_24[f"default_within_{LONG_HORIZON}m"].mean()},
    {"dataset": "valid_24", "n": len(valid_24), "default_rate":␣
 ↪valid_24[f"default_within_{LONG_HORIZON}m"].mean()},
    {"dataset": "test_24",  "n": len(test_24),  "default_rate":␣
 ↪test_24[f"default_within_{LONG_HORIZON}m"].mean()},
])
display(label_summary)
```

```
    dataset       n  default_rate
0  train_12  466345      0.051022
1  valid_12  421095      0.060556
2   test_12  434407      0.068378
3  train_24  466345      0.113403
4  valid_24  421095      0.133633
5   test_24  434302      0.136237
```

1.8 6. Features

These are deliberately simple and close to what the paper emphasizes: - grade / pricing proxy - purpose - housing - annual income - indebtedness - credit history - origination characteristics

[8]:
```
model_numeric = [
    "int_rate",
    "loan_amnt",
    "installment",
    "annual_inc",
    "dti",
    "delinq_2yrs",
    "inq_last_6mths",
    "open_acc",
    "pub_rec",
    "revol_bal",
    "revol_util",
    "total_acc",
    "mort_acc",
    "pub_rec_bankruptcies",
    "fico_avg",
    "credit_history_years",
    "term_num",
]

model_categorical = [
    "grade",
    "purpose",
    "home_ownership",
    "emp_length",
    "verification_status",
]

preprocessor = ColumnTransformer(
    transformers=[
        ("num", Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler()),
        ]), model_numeric),
        ("cat", Pipeline([
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("onehot", OneHotEncoder(handle_unknown="ignore")),
        ]), model_categorical),
    ]
)
```

1.9 7. Frequency-based models: LPM and Logit

This keeps the note-consistent standard model in place.

[9]:
```
def fit_frequency_models(train_df, valid_df, test_df, horizon):
    ycol = f"default_within_{horizon}m"
```

```
    X_train = train_df[model_numeric + model_categorical]
    y_train = train_df[ycol]
    X_valid = valid_df[model_numeric + model_categorical]
    y_valid = valid_df[ycol]
    X_test = test_df[model_numeric + model_categorical]
    y_test = test_df[ycol]

    lpm = Pipeline([
        ("prep", preprocessor),
        ("model", LinearRegression()),
    ])
    logit = Pipeline([
        ("prep", preprocessor),
        ("model", LogisticRegression(max_iter=2000, C=0.5,␣
↪random_state=RANDOM_STATE)),
    ])

    lpm.fit(X_train, y_train)
    logit.fit(X_train, y_train)

    lpm_valid_raw = lpm.predict(X_valid)
    lpm_test_raw = lpm.predict(X_test)
    lpm_valid = np.clip(lpm_valid_raw, 0, 1)
    lpm_test = np.clip(lpm_test_raw, 0, 1)

    logit_valid = logit.predict_proba(X_valid)[:, 1]
    logit_test = logit.predict_proba(X_test)[:, 1]

    results = pd.DataFrame([
        {
            "model": f"LPM_{horizon}m",
            "valid_roc_auc": roc_auc_score(y_valid, lpm_valid),
            "valid_pr_auc": average_precision_score(y_valid, lpm_valid),
            "valid_brier": brier_score_loss(y_valid, lpm_valid),
            "test_roc_auc": roc_auc_score(y_test, lpm_test),
            "test_pr_auc": average_precision_score(y_test, lpm_test),
            "test_brier": brier_score_loss(y_test, lpm_test),
        },
        {
            "model": f"Logit_{horizon}m",
            "valid_roc_auc": roc_auc_score(y_valid, logit_valid),
            "valid_pr_auc": average_precision_score(y_valid, logit_valid),
            "valid_brier": brier_score_loss(y_valid, logit_valid),
            "test_roc_auc": roc_auc_score(y_test, logit_test),
            "test_pr_auc": average_precision_score(y_test, logit_test),
            "test_brier": brier_score_loss(y_test, logit_test),
        },
```

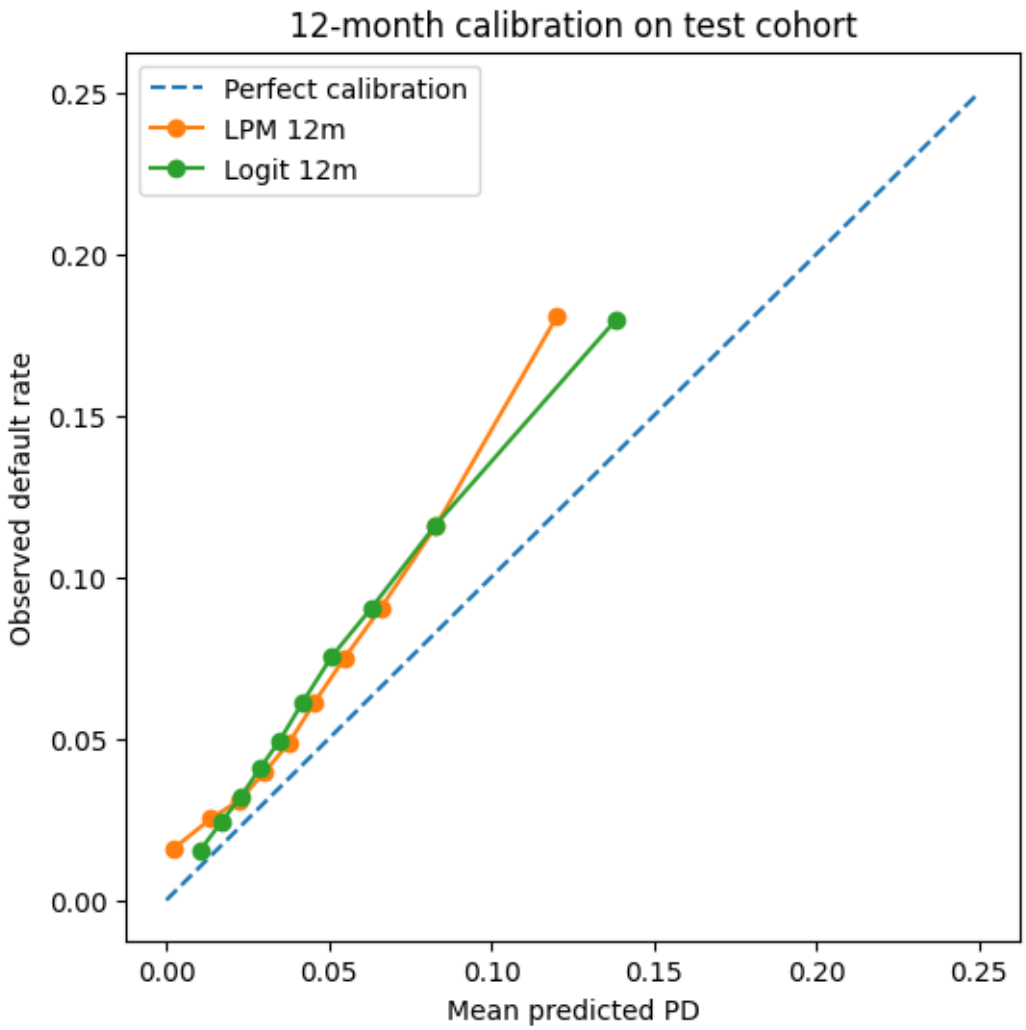
```
    ])

    payload = {
        "lpm": lpm,
        "logit": logit,
        "valid_pred_lpm": lpm_valid,
        "test_pred_lpm": lpm_test,
        "valid_pred_logit": logit_valid,
        "test_pred_logit": logit_test,
        "y_valid": y_valid,
        "y_test": y_test,
        "results": results,
    }
    return payload

freq12 = fit_frequency_models(train_12, valid_12, test_12, SHORT_HORIZON)
freq24 = fit_frequency_models(train_24, valid_24, test_24, LONG_HORIZON)

frequency_results = pd.concat(
    [
        freq12["results"].assign(horizon_months=SHORT_HORIZON),
        freq24["results"].assign(horizon_months=LONG_HORIZON),
    ],
    ignore_index=True,
)

display(frequency_results)
```

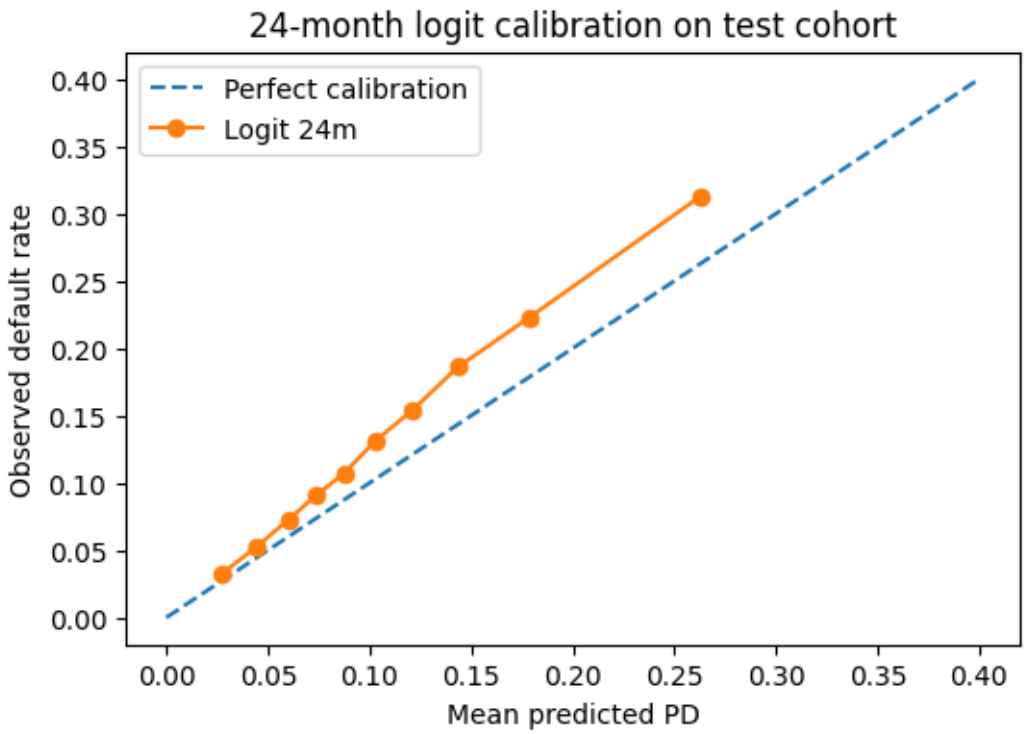
```
       model  valid_roc_auc  valid_pr_auc  valid_brier  test_roc_auc  \
0    LPM_12m       0.711923      0.136887     0.055189      0.702087
1  Logit_12m       0.711386      0.134783     0.055190      0.701331
2    LPM_24m       0.698317      0.255587     0.109941      0.693537
3  Logit_24m       0.697765      0.254254     0.109875      0.693798

   test_pr_auc  test_brier  horizon_months
0     0.149017    0.061983              12
1     0.146697    0.061915              12
2     0.258627    0.111897              24
3     0.257926    0.111711              24
```

1.9.1 7.1 Accuracy visuals for the frequency models

[10]:
```
# 12m calibration: LPM vs Logit
prob_true_lpm_12, prob_pred_lpm_12 = calibration_curve(freq12["y_test"],␣
 ↪freq12["test_pred_lpm"], n_bins=10, strategy="quantile")
prob_true_logit_12, prob_pred_logit_12 = calibration_curve(freq12["y_test"],␣
 ↪freq12["test_pred_logit"], n_bins=10, strategy="quantile")
```

```
plt.figure(figsize=(6, 6))
plt.plot([0, 0.25], [0, 0.25], linestyle="--", label="Perfect calibration")
plt.plot(prob_pred_lpm_12, prob_true_lpm_12, marker="o", label="LPM 12m")
plt.plot(prob_pred_logit_12, prob_true_logit_12, marker="o", label="Logit 12m")
plt.xlabel("Mean predicted PD")
plt.ylabel("Observed default rate")
plt.title("12-month calibration on test cohort")
plt.legend()
plt.show()

# 24m calibration: use the better standard frequency model (logit) against Cox␣
↪later
prob_true_logit_24, prob_pred_logit_24 = calibration_curve(freq24["y_test"],␣
↪freq24["test_pred_logit"], n_bins=10, strategy="quantile")
plt.figure(figsize=(6, 4))
plt.plot([0, 0.40], [0, 0.40], linestyle="--", label="Perfect calibration")
plt.plot(prob_pred_logit_24, prob_true_logit_24, marker="o", label="Logit 24m")
plt.xlabel("Mean predicted PD")
plt.ylabel("Observed default rate")
plt.title("24-month logit calibration on test cohort")
plt.legend()
plt.show()
```


12-month calibration on test cohort
Perfect calibration
LPM 12m
Logit 12m
Observed default rate
Mean predicted PD
0.00
0.05
0.10
0.15
0.20
0.25



1.10 8. Cox PH as the time-to-default extension

This is the industry-style survival model.

We fit it on the same paper-style train/valid/test vintages and then convert the survival curve into:
- a **24-month PD checkpoint** - an **average supported-horizon Cox curve**

[11]:
```
cox_numeric = model_numeric.copy()
cox_categorical = model_categorical.copy()

def prepare_cox_design(train_df, eval_df):
    train_term = train_df[["id", "duration_months", "default_event"] +␣
 ↪cox_numeric + cox_categorical].copy()
    eval_term = eval_df[["id", "duration_months", "default_event"] +␣
 ↪cox_numeric + cox_categorical].copy()

    for frame in [train_term, eval_term]:
        for c in cox_numeric + ["duration_months", "default_event"]:
            frame[c] = pd.to_numeric(frame[c], errors="coerce")
        for c in cox_categorical:
            frame[c] = frame[c].astype("string").fillna("Missing")

    train_term = train_term.dropna(subset=["duration_months", "default_event"]).
 ↪copy()
```

```
    eval_term = eval_term.dropna(subset=["duration_months", "default_event"]).
↪copy()

    train_medians = train_term[cox_numeric].median(numeric_only=True)
    train_term[cox_numeric] = train_term[cox_numeric].fillna(train_medians)
    eval_term[cox_numeric] = eval_term[cox_numeric].fillna(train_medians)

    train_dm = pd.get_dummies(train_term, columns=cox_categorical,␣
↪drop_first=True)
    eval_dm = pd.get_dummies(eval_term, columns=cox_categorical,␣
↪drop_first=True)

    feature_cols = [c for c in train_dm.columns if c not in ["id",␣
↪"duration_months", "default_event"]]

    for c in feature_cols:
        if c not in eval_dm.columns:
            eval_dm[c] = 0

    extra_eval = [c for c in eval_dm.columns if c not in ["id",␣
↪"duration_months", "default_event"] + feature_cols]
    if extra_eval:
        eval_dm = eval_dm.drop(columns=extra_eval)

    train_dm = train_dm[["id", "duration_months", "default_event"] +␣
↪feature_cols].copy()
    eval_dm = eval_dm[["id", "duration_months", "default_event"] +␣
↪feature_cols].copy()

    for frame in [train_dm, eval_dm]:
        bool_cols = frame.select_dtypes(include=["bool"]).columns
        if len(bool_cols) > 0:
            frame[bool_cols] = frame[bool_cols].astype(int)
        frame[feature_cols] = frame[feature_cols].astype(float)
        frame["duration_months"] = frame["duration_months"].astype(float)
        frame["default_event"] = frame["default_event"].astype(int)

    nunique = train_dm[feature_cols].nunique(dropna=False)
    constant_cols = nunique[nunique <= 1].index.tolist()
    if constant_cols:
        feature_cols = [c for c in feature_cols if c not in constant_cols]
        train_dm = train_dm[["id", "duration_months", "default_event"] +␣
↪feature_cols].copy()
        eval_dm = eval_dm[["id", "duration_months", "default_event"] +␣
↪feature_cols].copy()
```

```
    return train_dm, eval_dm, feature_cols

cox_train, cox_valid, cox_features = prepare_cox_design(train_loans,␣
 ↪valid_loans)
_, cox_test, _ = prepare_cox_design(train_loans, test_loans)

cph = CoxPHFitter(penalizer=0.1)
cph.fit(
    cox_train.drop(columns=["id"]),
    duration_col="duration_months",
    event_col="default_event",
    show_progress=False,
)

valid_risk = cph.predict_partial_hazard(cox_valid[cox_features]).values.ravel()
test_risk = cph.predict_partial_hazard(cox_test[cox_features]).values.ravel()

cox_c_valid = concordance_index(
    cox_valid["duration_months"],
    -valid_risk,
    cox_valid["default_event"],
)
cox_c_test = concordance_index(
    cox_test["duration_months"],
    -test_risk,
    cox_test["default_event"],
)

cox_summary = pd.DataFrame([{
    "model": "Cox_PH_time_to_default",
    "valid_c_index": cox_c_valid,
    "test_c_index": cox_c_test,
    "n_train": len(cox_train),
    "n_valid": len(cox_valid),
    "n_test": len(cox_test),
}])

display(cox_summary)
```

```
                    model  valid_c_index  test_c_index  n_train  n_valid  \
0  Cox_PH_time_to_default       0.685673      0.685001   466345   421095

   n_test
0  434407
```

1.10.1 8.1 The strongest Cox hazard ratios

[12]:
```
cox_coef_view = cph.summary[["coef", "exp(coef)", "se(coef)", "z", "p"]].
 ↪sort_values("exp(coef)", ascending=False)
display(cox_coef_view.head(20))
```

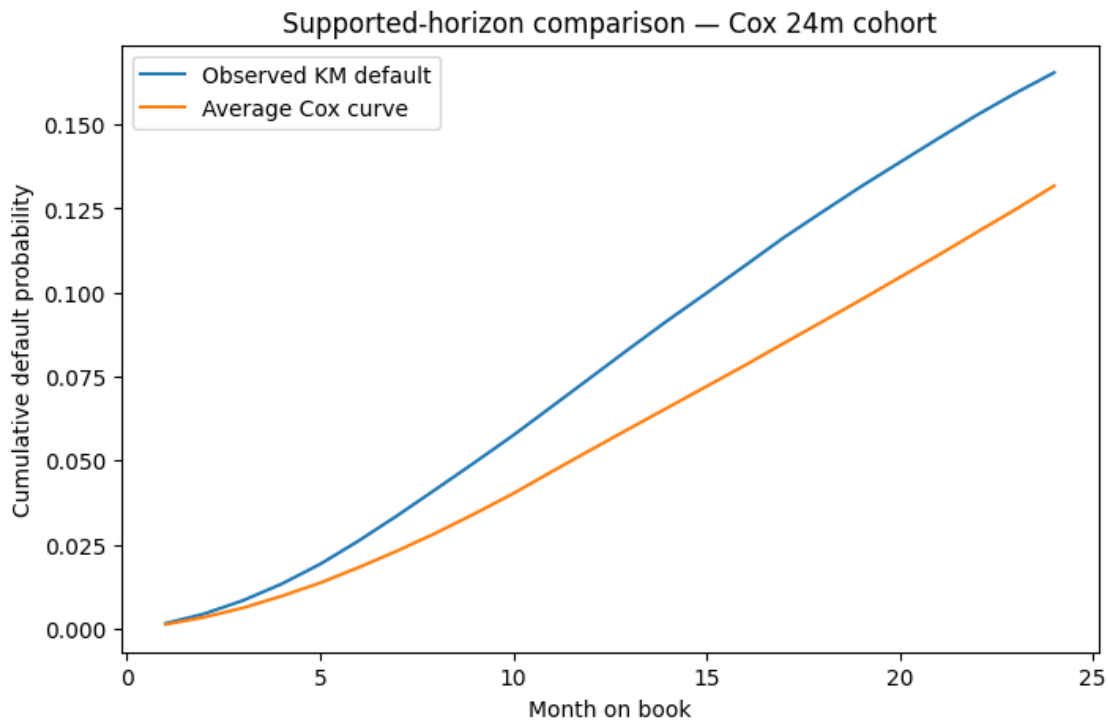
```
                                       coef  exp(coef)  se(coef)          z  \
covariate
grade_G                            0.327748   1.387840  0.027008  12.135345
purpose_small_business             0.289166   1.335313  0.021408  13.507466
grade_F                            0.266647   1.305580  0.015510  17.191774
grade_E                            0.245254   1.277946  0.010704  22.912775
grade_D                            0.184431   1.202534  0.008228  22.415839
emp_length_Missing                 0.176933   1.193551  0.013567  13.041089
purpose_educational                0.174224   1.190323  0.090512   1.924884
purpose_renewable_energy           0.129382   1.138124  0.101379   1.276221
home_ownership_OTHER               0.126533   1.134887  0.136350   0.927997
grade_C                            0.092935   1.097390  0.007151  12.995780
purpose_moving                     0.077684   1.080781  0.034387   2.259090
purpose_medical                    0.070724   1.073285  0.028658   2.467877
inq_last_6mths                     0.066382   1.068635  0.002498  26.574365
purpose_house                      0.062333   1.064316  0.040383   1.543523
home_ownership_RENT                0.061264   1.063179  0.007255   8.444225
pub_rec_bankruptcies               0.058406   1.060145  0.009286   6.289333
verification_status_Source Verified 0.054143  1.055636  0.006574   8.236564
int_rate                           0.039760   1.040561  0.000828  48.020705
purpose_debt_consolidation         0.039603   1.040398  0.007054   5.614187
emp_length_< 1 year                0.026845   1.027209  0.010994   2.441776

                                            p
covariate
grade_G                          6.862024e-34
purpose_small_business           1.413014e-41
grade_F                          3.060243e-66
grade_E                         3.465535e-116
grade_D                         2.757953e-111
emp_length_Missing               7.142965e-39
purpose_educational              5.424384e-02
purpose_renewable_energy         2.018775e-01
home_ownership_OTHER             3.534089e-01
grade_C                          1.292834e-38
purpose_moving                   2.387778e-02
purpose_medical                  1.359171e-02
inq_last_6mths                  1.343248e-155
purpose_house                    1.227038e-01
home_ownership_RENT              3.060689e-17
pub_rec_bankruptcies             3.188323e-10
verification_status_Source Verified 1.772262e-16
```

```
int_rate                                0.000000e+00
purpose_debt_consolidation              1.974882e-08
emp_length_< 1 year                     1.461523e-02
```

1.10.2 8.2 Supported-horizon Cox curve check

[13]:
```
def km_curve(loans, max_horizon=None):
    kmf = KaplanMeierFitter()
    kmf.fit(
        durations=loans["duration_months"],
        event_observed=loans["default_event"],
    )
    surv = kmf.survival_function_.reset_index()
    surv.columns = ["month_on_book", "survival"]
    surv["month_on_book"] = surv["month_on_book"].astype(float).round().
 ↪astype(int)
    surv["cum_default_km"] = 1 - surv["survival"]
    if max_horizon is not None:
        surv = surv[surv["month_on_book"] <= max_horizon].copy()
    return surv

km_test = km_curve(test_loans, max_horizon=LONG_HORIZON)

surv_pred_test = cph.predict_survival_function(cox_test[cox_features])
avg_surv = surv_pred_test.mean(axis=1).reset_index()
avg_surv.columns = ["month_on_book", "survival_cox_avg"]
avg_surv["month_on_book"] = avg_surv["month_on_book"].astype(float).round().
 ↪astype(int)
avg_surv = avg_surv.groupby("month_on_book",␣
 ↪as_index=False)["survival_cox_avg"].last()
avg_surv["cum_default_cox_avg"] = 1 - avg_surv["survival_cox_avg"]

compare_curve = km_test.merge(
    avg_surv[["month_on_book", "cum_default_cox_avg"]],
    on="month_on_book",
    how="inner",
)

plt.figure(figsize=(8, 5))
plt.plot(compare_curve["month_on_book"], compare_curve["cum_default_km"],␣
 ↪label="Observed KM default")
plt.plot(compare_curve["month_on_book"], compare_curve["cum_default_cox_avg"],␣
 ↪label="Average Cox curve")
plt.xlabel("Month on book")
plt.ylabel("Cumulative default probability")
plt.title(f"Supported-horizon comparison - Cox {LONG_HORIZON}m cohort")
plt.legend()
```

```
plt.show()
```



1.11 9. Convert Cox into a simple 24-month PD and compare it with 24-month logit

This is the key practical comparison: - **standard frequency model**: 24m logit - **industry-style survival model**: 24m PD implied by Cox

[14]:
```
# 24m PD from Cox on the validation/test sets
cox_valid_24 = cph.predict_survival_function(cox_valid[cox_features],␣
 ↪times=[LONG_HORIZON]).T
cox_valid_24.columns = ["survival_24m"]
cox_valid_24["pd_24m_cox"] = 1 - cox_valid_24["survival_24m"]
cox_valid_24 = cox_valid_24.reset_index(drop=True)
cox_valid_24["id"] = cox_valid["id"].values

cox_test_24 = cph.predict_survival_function(cox_test[cox_features],␣
 ↪times=[LONG_HORIZON]).T
cox_test_24.columns = ["survival_24m"]
cox_test_24["pd_24m_cox"] = 1 - cox_test_24["survival_24m"]
cox_test_24 = cox_test_24.reset_index(drop=True)
cox_test_24["id"] = cox_test["id"].values
```

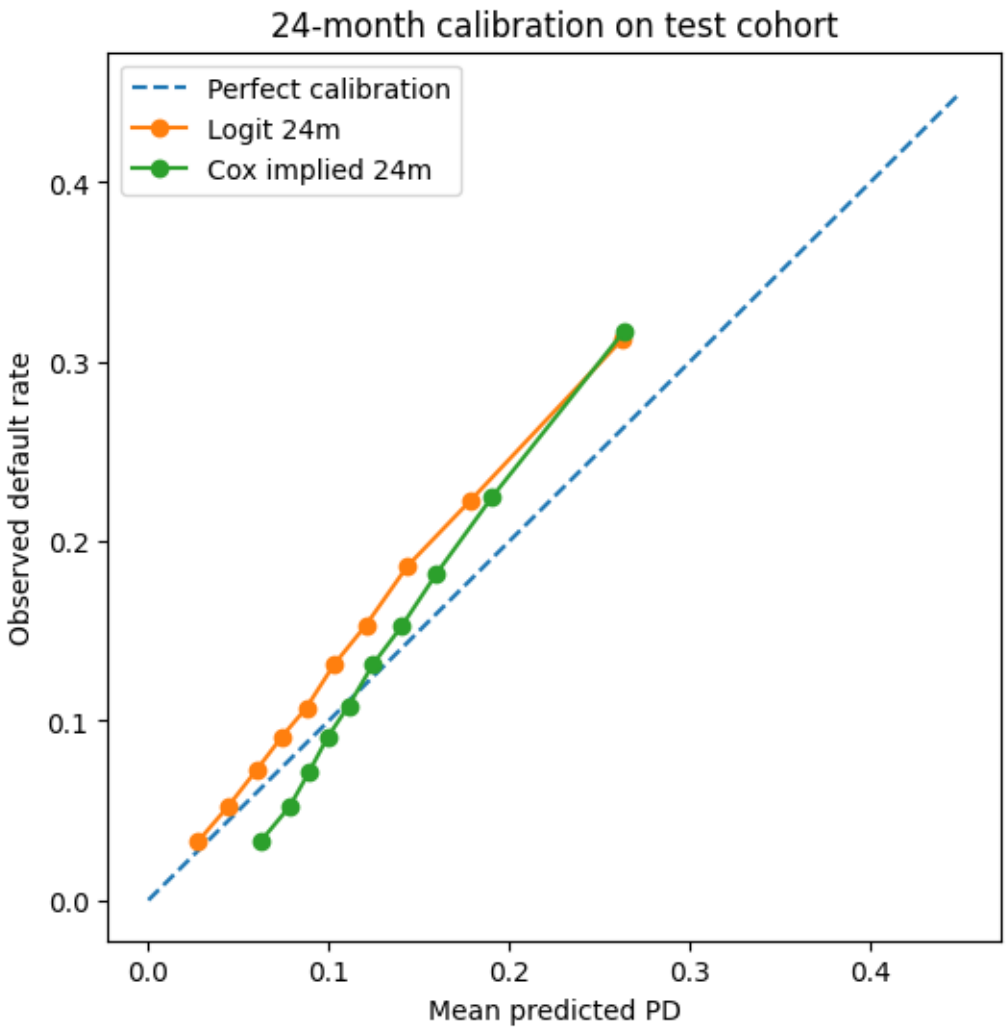
```
valid24_compare = valid_24[["id", f"default_within_{LONG_HORIZON}m"]].merge(
    cox_valid_24[["id", "pd_24m_cox"]],
    on="id",
    how="inner",
)
test24_compare = test_24[["id", f"default_within_{LONG_HORIZON}m"]].merge(
    cox_test_24[["id", "pd_24m_cox"]],
    on="id",
    how="inner",
)

valid24_compare["pd_24m_logit"] = freq24["valid_pred_logit"]
test24_compare["pd_24m_logit"] = freq24["test_pred_logit"]

y_valid_24 = valid24_compare[f"default_within_{LONG_HORIZON}m"]
y_test_24 = test24_compare[f"default_within_{LONG_HORIZON}m"]

longterm_results = pd.DataFrame([
    {
        "model": "Logit_24m",
        "valid_roc_auc": roc_auc_score(y_valid_24,␣
 ↪valid24_compare["pd_24m_logit"]),
        "valid_pr_auc": average_precision_score(y_valid_24,␣
 ↪valid24_compare["pd_24m_logit"]),
        "valid_brier": brier_score_loss(y_valid_24,␣
 ↪valid24_compare["pd_24m_logit"]),
        "test_roc_auc": roc_auc_score(y_test_24,␣
 ↪test24_compare["pd_24m_logit"]),
        "test_pr_auc": average_precision_score(y_test_24,␣
 ↪test24_compare["pd_24m_logit"]),
        "test_brier": brier_score_loss(y_test_24,␣
 ↪test24_compare["pd_24m_logit"]),
    },
    {
        "model": "Cox_implied_PD_24m",
        "valid_roc_auc": roc_auc_score(y_valid_24,␣
 ↪valid24_compare["pd_24m_cox"]),
        "valid_pr_auc": average_precision_score(y_valid_24,␣
 ↪valid24_compare["pd_24m_cox"]),
        "valid_brier": brier_score_loss(y_valid_24,␣
 ↪valid24_compare["pd_24m_cox"]),
        "test_roc_auc": roc_auc_score(y_test_24, test24_compare["pd_24m_cox"]),
        "test_pr_auc": average_precision_score(y_test_24,␣
 ↪test24_compare["pd_24m_cox"]),
        "test_brier": brier_score_loss(y_test_24, test24_compare["pd_24m_cox"]),
    },
```

```
])

display(longterm_results)
```

```
                model  valid_roc_auc  valid_pr_auc  valid_brier  test_roc_auc  \
0           Logit_24m       0.697765      0.254254     0.109875      0.693798
1  Cox_implied_PD_24m       0.699143      0.257015     0.109599      0.695161

   test_pr_auc  test_brier
0     0.257926    0.111711
1     0.260684    0.111295
```

1.11.1 9.1 Calibration: 24m logit vs 24m Cox

[15]:
```
prob_true_logit_24, prob_pred_logit_24 = calibration_curve(
    y_test_24, test24_compare["pd_24m_logit"], n_bins=10, strategy="quantile"
)
prob_true_cox_24, prob_pred_cox_24 = calibration_curve(
    y_test_24, test24_compare["pd_24m_cox"], n_bins=10, strategy="quantile"
)

plt.figure(figsize=(6, 6))
plt.plot([0, 0.45], [0, 0.45], linestyle="--", label="Perfect calibration")
plt.plot(prob_pred_logit_24, prob_true_logit_24, marker="o", label="Logit 24m")
plt.plot(prob_pred_cox_24, prob_true_cox_24, marker="o", label="Cox implied␣
 ↪24m")
plt.xlabel("Mean predicted PD")
plt.ylabel("Observed default rate")
plt.title("24-month calibration on test cohort")
plt.legend()
plt.show()
```



1.12 10. Choose the simplest long-term model

Rule: - choose the lower **validation Brier score** - if nearly tied, prefer the simpler standard model (**Logit 24m**)

[16]:
```
lt = longterm_results.set_index("model")
brier_gap = lt.loc["Cox_implied_PD_24m", "valid_brier"] - lt.loc["Logit_24m",␣
↪"valid_brier"]

if brier_gap < -0.001:
    chosen_longterm_model = "Cox_implied_PD_24m"
else:
    chosen_longterm_model = "Logit_24m"

selection_table = longterm_results.copy()
```

```
selection_table["chosen_for_simple_longterm"] = selection_table["model"].
 ↪eq(chosen_longterm_model)
display(selection_table)

print("Chosen simple long-term PD model:", chosen_longterm_model)
```

```
                model  valid_roc_auc  valid_pr_auc  valid_brier  test_roc_auc  \
0           Logit_24m       0.697765      0.254254     0.109875      0.693798
1  Cox_implied_PD_24m       0.699143      0.257015     0.109599      0.695161

   test_pr_auc  test_brier  chosen_for_simple_longterm
0     0.257926    0.111711                        True
1     0.260684    0.111295                       False

Chosen simple long-term PD model: Logit_24m
```

1.13 11. Refit final production-style models on train + validation

This gives teammates one clean set of PD outputs.

[17]:
```
# Combine train+valid for final fitting
tv_loans = pd.concat([train_loans, valid_loans], ignore_index=True)
tv_12 = pd.concat([train_12, valid_12], ignore_index=True)
tv_24 = pd.concat([train_24, valid_24], ignore_index=True)

# Final 12m logit
X_tv_12 = tv_12[model_numeric + model_categorical]
y_tv_12 = tv_12[f"default_within_{SHORT_HORIZON}m"]

final_logit_12 = Pipeline([
    ("prep", preprocessor),
    ("model", LogisticRegression(max_iter=2000, C=0.5,␣
 ↪random_state=RANDOM_STATE)),
])
final_logit_12.fit(X_tv_12, y_tv_12)

# Final 24m logit
X_tv_24 = tv_24[model_numeric + model_categorical]
y_tv_24 = tv_24[f"default_within_{LONG_HORIZON}m"]

final_logit_24 = Pipeline([
    ("prep", preprocessor),
    ("model", LogisticRegression(max_iter=2000, C=0.5,␣
 ↪random_state=RANDOM_STATE)),
])
final_logit_24.fit(X_tv_24, y_tv_24)

# Final Cox
```

```
final_cox_train, final_cox_all, final_cox_features =␣
↪prepare_cox_design(tv_loans, df_model)
final_cph = CoxPHFitter(penalizer=0.1)
final_cph.fit(
    final_cox_train.drop(columns=["id"]),
    duration_col="duration_months",
    event_col="default_event",
    show_progress=False,
)
```

[17]: <lifelines.CoxPHFitter: fitted with 887440 total observations, 734375 right-censored observations>

1.14 12. Score the whole modeled portfolio and export one teammate-ready table

[18]:
```
pd_output = df_model[[
    "id", "issue_d", "loan_status", "grade", "int_rate", "loan_amnt",
    "annual_inc", "dti", "fico_avg", "term_num"
]].copy()

X_all = df_model[model_numeric + model_categorical]
pd_output["pd_12m_logit"] = final_logit_12.predict_proba(X_all)[:, 1]
pd_output["pd_24m_logit"] = final_logit_24.predict_proba(X_all)[:, 1]

cox_all_24 = final_cph.
↪predict_survival_function(final_cox_all[final_cox_features],␣
↪times=[LONG_HORIZON]).T
cox_all_24.columns = ["survival_24m"]
cox_all_24["pd_24m_cox"] = 1 - cox_all_24["survival_24m"]
cox_all_24 = cox_all_24.reset_index(drop=True)
cox_all_24["id"] = final_cox_all["id"].values

pd_output = pd_output.merge(cox_all_24[["id", "pd_24m_cox"]], on="id",␣
↪how="left")

if chosen_longterm_model == "Cox_implied_PD_24m":
    pd_output["pd_24m_chosen"] = pd_output["pd_24m_cox"]
else:
    pd_output["pd_24m_chosen"] = pd_output["pd_24m_logit"]

pd_output["chosen_longterm_model"] = chosen_longterm_model

display(pd_output.head())

pd_output.to_csv(OUTPUT_DIR / "pd_predictions_for_teammates.csv", index=False)
```

```
frequency_results.to_csv(OUTPUT_DIR / "frequency_model_metrics.csv",␣
 ↪index=False)
longterm_results.to_csv(OUTPUT_DIR / "longterm_model_comparison.csv",␣
 ↪index=False)
cox_summary.to_csv(OUTPUT_DIR / "cox_time_to_default_summary.csv", index=False)

print("Saved:")
print("-", OUTPUT_DIR / "pd_predictions_for_teammates.csv")
print("-", OUTPUT_DIR / "frequency_model_metrics.csv")
print("-", OUTPUT_DIR / "longterm_model_comparison.csv")
print("-", OUTPUT_DIR / "cox_time_to_default_summary.csv")
```

```
         id    issue_d loan_status grade  int_rate  loan_amnt  annual_inc  \
0  68407277 2015-12-01  Fully Paid     C     13.99     3600.0     55000.0
1  68355089 2015-12-01  Fully Paid     C     11.99    24700.0     65000.0
2  68341763 2015-12-01  Fully Paid     B     10.78    20000.0     63000.0
3  66310712 2015-12-01     Current     C     14.85    35000.0    110000.0
4  68476807 2015-12-01  Fully Paid     F     22.45    10400.0    104433.0

     dti  fico_avg  term_num  pd_12m_logit  pd_24m_logit  pd_24m_cox  \
0   5.91     677.0      36.0      0.043247      0.102377    0.130483
1  16.06     717.0      36.0      0.084606      0.190163    0.173624
2  10.78     697.0      60.0      0.019489      0.054399    0.081046
3  17.06     787.0      60.0      0.038070      0.091440    0.124820
4  25.37     697.0      60.0      0.142994      0.235424    0.247683

   pd_24m_chosen chosen_longterm_model
0       0.102377             Logit_24m
1       0.190163             Logit_24m
2       0.054399             Logit_24m
3       0.091440             Logit_24m
4       0.235424             Logit_24m

Saved:
- data/processed_paper_style_pd/pd_predictions_for_teammates.csv
- data/processed_paper_style_pd/frequency_model_metrics.csv
- data/processed_paper_style_pd/longterm_model_comparison.csv
- data/processed_paper_style_pd/cox_time_to_default_summary.csv
```